

Práctica de laboratorio

507-EGSánchez-PARP503-PingARP.docx

Objetivo de la práctica

Utilizar distintas herramientas estándar en los sistemas operativos de red para la pila de protocolos TCP/IP. Instalaremos un sniffer (Wireshark, por ejemplo) en cada uno de estos dos sistemas que describiremos a continuación para capturar todo el tráfico generado durante la ejecución de la práctica.

Inventario de material necesario

Dos máquinas con Windows instalado (en mi caso, un Ubuntu server y un Ubuntu desktop).

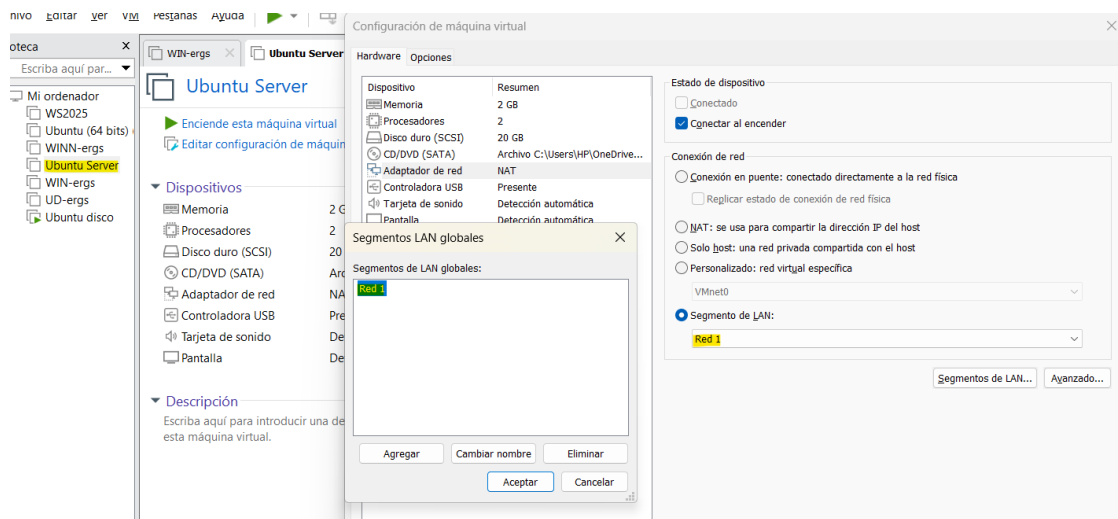
Necesitaremos al menos dos nodos de red ejecutando TCP/IP, que denominaremos ergs1 (pc1) y ergs2 (pc2), que pueden ser Windows o Linux o mezclas de ambos.

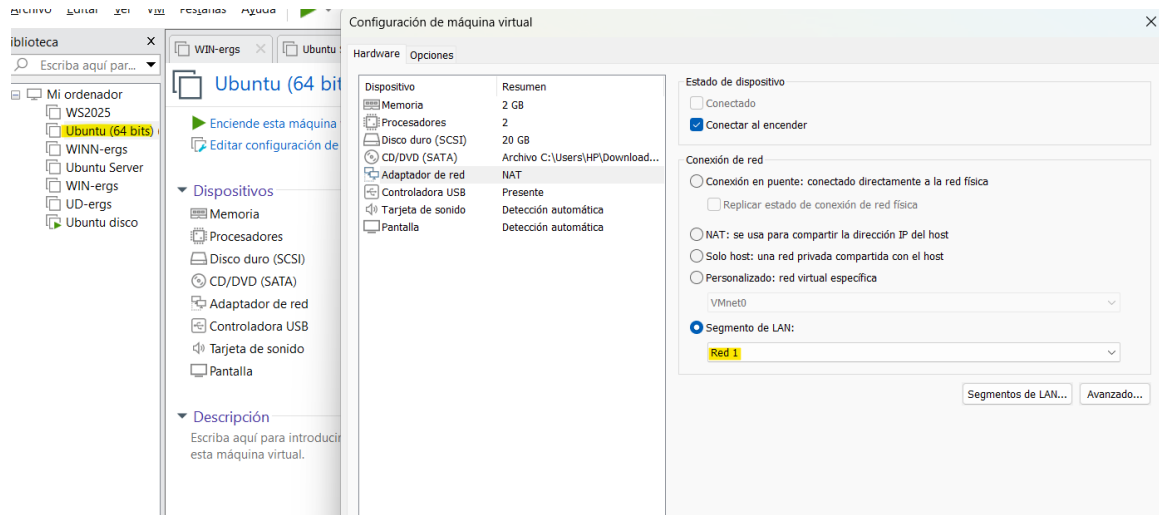
También debemos tener a mano las direcciones IP (nivel 3) y MAC (nivel 2 o direcciones físicas) de cada una de las máquinas que intervengan en la práctica y las direcciones IP de las máquinas deben estar en la misma subred.

En el caso de que estén en distinta subred, usaremos un enrutador que interconecte las dos subredes.

Ejecución

Para asegurarnos que las máquinas se puedan comunicar entre sí más adelante, cambiamos la conexión de red del adaptador a “segmento de LAN” y crearemos una llamada “Red 1” a la que ambas estarán conectadas.





En Ubuntu server (ergs1)

Aquí asignaremos la ip manualmente con el comando “*sudo ip addr add 192.168.10.1/24 dev ens33*”

Para ver la ip y la dirección MAC de la máquina hacemos “*ip a*”

```
ergs@ub-srv:~$ sudo ip addr add 192.168.10.1/24 dev ens33
[sudo] password for ergs:
ergs@ub-srv:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:8b:ab:59 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.1/24 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe8b:ab59/64 scope link
        valid_lft forever preferred_lft forever
ergs@ub-srv:~$
```

Observamos que nuestra ip en ergs1 es 192.168.10.1/24 y la dirección MAC 00:0c:29:8b:ab:59

En Ubuntu desktop (ergs2)

Ejecutamos el comando “*ip a*” para mostrar la ip y dirección MAC actuales de máquina:

```
ergs@UBD64-VMware-Virtual-Platform: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ergs@UBD64-VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:76:ff:11 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.107.128/24 scope global dynamic noprefixroute ens33
        valid_lft 1698sec preferred_lft 1698sec
    inet6 fe80::20c:29ff:fe76:ff11/64 scope link
        valid_lft forever preferred_lft forever
ergs@UBD64-VMware-Virtual-Platform:~$
```

Inicialmente esta ip se encuentra en la misma subred que ergs1, así que no podrá comunicarse con ella. Para evitar usar un enrutador virtual para conectarlas, cambiaremos de forma similar a la que hicimos antes la ip de ergs2 para que esté en la misma subred con los comandos

“`sudo ip addr flush dev ens33`” para borrar la ip anterior y

“`sudo ip addr add 192.168.10.2/24 dev ens33`” para asignar la nueva ip que pertenece a la misma subred.

```
ergs@UBD64-VMware-Virtual-Platform:~$ sudo ip addr flush dev ens33
ergs@UBD64-VMware-Virtual-Platform:~$ sudo ip addr add 192.168.10.2/24 dev ens33
ergs@UBD64-VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:76:ff:11 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.2/24 scope global ens33
        valid_lft forever preferred_lft forever
ergs@UBD64-VMware-Virtual-Platform:~$
```

Para poder hacer la escucha en esta práctica estaremos usando “Wireshark” en ergs2 y la versión de terminal “tshark” en ergs1.

```
ergs@ub-srv:~$ sudo apt install tshark -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
  tshark
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 80 no actualizados.
Se necesita descargar 158 kB de archivos.
Se utilizarán 416 kB de espacio de disco adicional después de esta operación.
Des:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 tshark amd64 4.2.2-1.1build3 [158
Descargados 158 kB en 0s (750 kB/s)
Seleccionando el paquete tshark previamente no seleccionado.
(Leyendo la base de datos ... 100205 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../tshark_4.2.2-1.1build3_amd64.deb ...
Desempaquetando tshark (4.2.2-1.1build3) ...
Configurando tshark (4.2.2-1.1build3) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

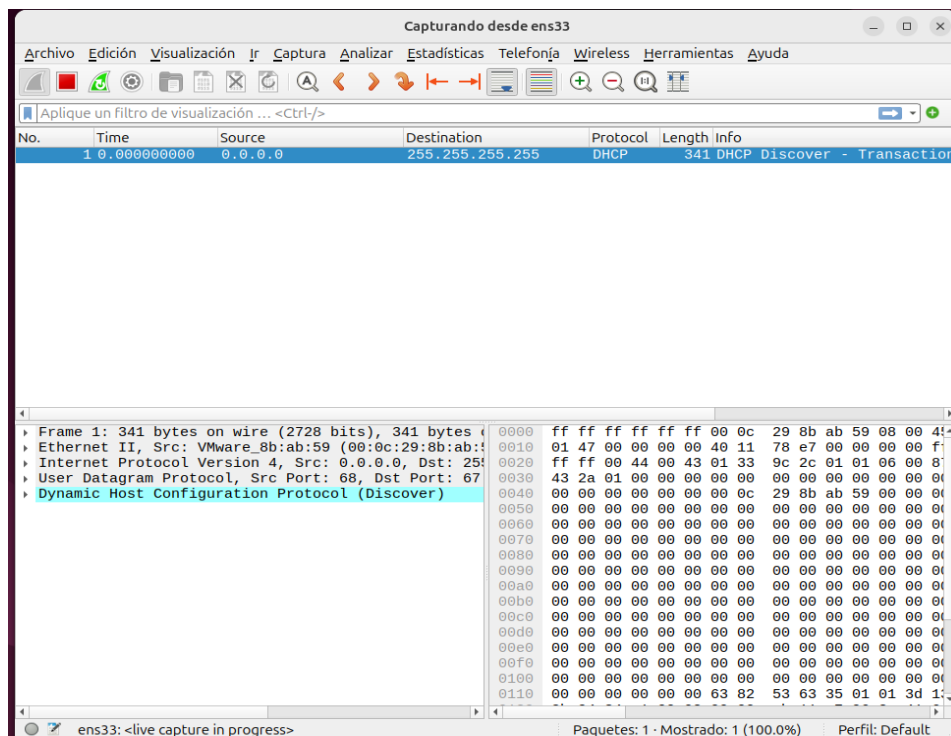
Restarting services...

Service restarts being deferred:
  systemctl restart unattended-upgrades.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ergs@ub-srv:~$ sudo tshark -i ens33 -f "arp or icmp"
Running as user "root" and group "root". This could be dangerous.
Capturing on 'ens33'
```



Operación con la caché de ARP

Consultamos el estado inicial de la caché de ARP con “*arp -a*” tanto en ergs1 como en ergs2

ergs2

```
ergs@UBD64-VMware-Virtual-Platform:~$ arp -a
ergs@UBD64-VMware-Virtual-Platform:~$
```

ergs1

```
ergs@ub-srv:~$ arp -a
ergs@ub-srv:~$
```

Al no salir nada, entendemos que es porque no tenemos nada en la caché, así que estaremos trabajando en un entorno limpio.

Ahora, para configurar la tabla de rutas para ambas tarjetas ejecutamos el comando “*sudo ip route add 192.168.10.0/24 dev ens33*” para añadir manualmente la subred que deben reconocer. Para confirmar que haya salido bien la operación vemos la tabla de rutas con el comando “*ip route*”

ergs2

```
ergs@UBD64-VMware-Virtual-Platform:~$ sudo ip route add 192.168.10.0/24 dev ens33
ergs@UBD64-VMware-Virtual-Platform:~$ ip route
192.168.10.0/24 dev ens33 scope link
```

ergs1

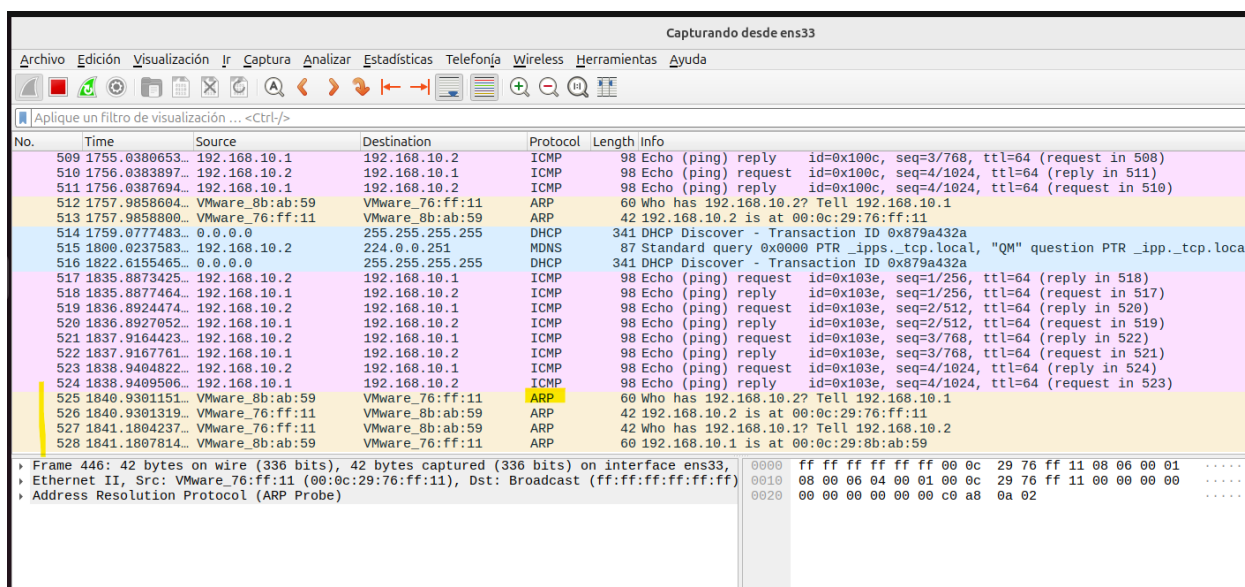
```
ergs@ub-srv:~$ sudo ip route add 192.168.10.0/24 dev ens33
[sudo] password for ergs:
RTNETLINK answers: File exists
ergs@ub-srv:~$ ip route
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.1
ergs@ub-srv:~$
```

Hacemos ping a ergs1 mandando 4 paquetes desde nuestro Ubuntu desktop ergs2 con el comando “ping -c 4 192.168.10.1”

```
ergs@UBD64-VMware-Virtual-Platform:~$ ping -c 4 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=0.534 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=0.623 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=0.855 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=64 time=0.403 ms

--- 192.168.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3088ms
rtt min/avg/max/mdev = 0.403/0.603/0.855/0.164 ms
ergs@UBD64-VMware-Virtual-Platform:~$
```

Abrimos Wireshark y veremos la secuencia completa de comunicación entre los dos nodos de la misma subred (192.168.10.0/24) tras ejecutar el ping:



No.	Time	Source	Destination	Protocol	Length	Info
509	1755.0380653...	192.168.10.1	192.168.10.2	ICMP	98	Echo (ping) reply id=0x100c, seq=3/768, ttl=64 (request in 508)
510	1756.0383897...	192.168.10.2	192.168.10.1	ICMP	98	Echo (ping) request id=0x100c, seq=4/1024, ttl=64 (reply in 511)
511	1756.0387694...	192.168.10.1	192.168.10.2	ICMP	98	Echo (ping) reply id=0x100c, seq=4/1024, ttl=64 (request in 510)
512	1757.9858604...	VMware_8b:ab:59	VMware_76:ff:11	ARP	60	Who has 192.168.10.2? Tell 192.168.10.1
513	1757.9858800...	VMware_76:ff:11	VMware_8b:ab:59	ARP	42	192.168.10.2 is at 00:0c:29:76:ff:11
514	1759.0777483...	0.0.0.0	255.255.255.255	DHCP	341	DHCP Discover - Transaction ID 0x879a432a
515	1800.0237583...	192.168.10.2	224.0.0.251	MDNS	87	Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipp._tcp.local
516	1822.6155465...	0.0.0.0	255.255.255.255	DHCP	341	DHCP Discover - Transaction ID 0x879a432a
517	1835.8873425...	192.168.10.2	192.168.10.1	ICMP	98	Echo (ping) request id=0x103e, seq=1/256, ttl=64 (reply in 518)
518	1835.8877464...	192.168.10.1	192.168.10.2	ICMP	98	Echo (ping) reply id=0x103e, seq=1/256, ttl=64 (request in 517)
519	1836.8924474...	192.168.10.2	192.168.10.1	ICMP	98	Echo (ping) request id=0x103e, seq=2/512, ttl=64 (reply in 520)
520	1836.8927052...	192.168.10.1	192.168.10.2	ICMP	98	Echo (ping) reply id=0x103e, seq=2/512, ttl=64 (request in 519)
521	1837.9164423...	192.168.10.2	192.168.10.1	ICMP	98	Echo (ping) request id=0x103e, seq=3/768, ttl=64 (reply in 522)
522	1837.9167761...	192.168.10.1	192.168.10.2	ICMP	98	Echo (ping) reply id=0x103e, seq=3/768, ttl=64 (request in 521)
523	1838.9404822...	192.168.10.2	192.168.10.1	ICMP	98	Echo (ping) request id=0x103e, seq=4/1024, ttl=64 (reply in 524)
524	1838.9409566...	192.168.10.1	192.168.10.2	ICMP	98	Echo (ping) reply id=0x103e, seq=4/1024, ttl=64 (request in 523)
525	1840.9301151...	VMware_8b:ab:59	VMware_76:ff:11	ARP	60	Who has 192.168.10.2? Tell 192.168.10.1
526	1840.9301319...	VMware_76:ff:11	VMware_8b:ab:59	ARP	42	192.168.10.2 is at 00:0c:29:76:ff:11
527	1841.1804237...	VMware_76:ff:11	VMware_8b:ab:59	ARP	42	Who has 192.168.10.1? Tell 192.168.10.2
528	1841.1807814...	VMware_8b:ab:59	VMware_76:ff:11	ARP	60	192.168.10.1 is at 00:0c:29:8b:ab:59

Estos bloques amarillos representan la fase de la resolución de direcciones En el paquete 527 el origen (.10.2) emite un mensaje de broadcast preguntando quién tiene la IP acaba en 10.1 Luego en el paquete seguido 528, el destino responde de forma directa (unicast) indicando su dirección MAC real.

Comprobamos que la caché de ambas máquinas ha cambiado:

ergs1

```
ergs@ub-srv:~$ arp -a
? (192.168.10.2) at 00:0c:29:76:ff:11 [ether] on ens33
ergs@ub-srv:~$
```

ergs2

```
ergs@UBD64-VMware-Virtual-Platform:~$ arp -a
? (192.168.10.1) en 00:0c:29:8b:ab:59 [ether] en ens33
ergs@UBD64-VMware-Virtual-Platform:~$
```

Esto se debe a que, por una parte, el emisor (ergs2) actualiza su tabla para poder encapsular los datagramas IP en tramas Ethernet dirigidas a una MAC específica. Por otro lado, el receptor (ergs1) realiza una actualización cruzada al recibir la petición, almacenando los datos del emisor para agilizar la respuesta (Reply) inmediata. Este comportamiento minimiza el uso de transmisiones broadcast en la red, optimizando el ancho de banda.

Para recuperar el registro de ergs2 en la caché ARP de ergs1 habiéndola borrado, podemos escribir en la terminal de ergs1 el comando “*ping -c 1 192.168.10.2*”. Esto hace que el intentar enviar el ping, ergs1 consulte el caché, y al ver que no tiene la MAC del ergs2, lanza automáticamente un “ARP REQUEST” al cable. Después de que ergs2 responde el registro se crea de nuevo de forma dinámica.

Operación con ARP

Ahora haremos ping desde ergs1 a una dirección que no existe en nuestra red, en este caso “192.168.10.50”.

```
ergs@ub-srv:~$ ping 192.168.10.50
PING 192.168.10.50 (192.168.10.50) 56(84) bytes of data.
From 192.168.10.1 icmp_seq=1 Destination Host Unreachable
From 192.168.10.1 icmp_seq=2 Destination Host Unreachable
From 192.168.10.1 icmp_seq=3 Destination Host Unreachable
From 192.168.10.1 icmp_seq=4 Destination Host Unreachable
From 192.168.10.1 icmp_seq=5 Destination Host Unreachable
From 192.168.10.1 icmp_seq=6 Destination Host Unreachable
^C
--- 192.168.10.50 ping statistics ---
8 packets transmitted, 0 received, +6 errors, 100% packet loss, time 7167ms
pipe 4
ergs@ub-srv:~$
```

Como vemos, el destino de la red es inaccesible y no se recibió ningún paquete. A la hora de comprobar en la caché, vemos que se intentó mandar algo a la dirección IP anterior (la cual no estaba asociada a ninguna dirección Mac) desde nuestra tarjeta ens33.

```
ergs@ub-srv:~$ arp -a
? (192.168.10.2) at 00:0c:29:76:ff:11 [ether] on ens33
? (192.168.10.50) at <incomplete> on ens33
ergs@ub-srv:~$
```

Para añadirle una dirección Mac a esta IP ficticia usaremos el comando

“*sudo arp -s 192.168.10. 50 00:00:00:01:02:03*”.

Si comprobamos ahora la caché con “arp -a” veremos que en vez de <incomplete> saldrá la MAC que nosotros queríamos:

```
ergs@ub-srv:~$ sudo arp -s 192.168.10.50 00:00:00:01:02:03
ergs@ub-srv:~$ arp -a
a: Host name lookup failure
ergs@ub-srv:~$ arp -a
? (192.168.10.2) at 00:0c:29:76:ff:11 [ether] on ens33
? (192.168.10.50) at 00:00:00:01:02:03 [ether] PERM on ens33
ergs@ub-srv:~$
```

Volvemos a hacer ping a la dirección ip ficticia para ver que cambia.

```

ergs@ub-srv:~$ ping -c 4 192.168.10.50
PING 192.168.10.50 (192.168.10.50) 56(84) bytes of data.
--- 192.168.10.50 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3065ms

ergs@ub-srv:~$ arp -a
? (192.168.10.2) at 00:0c:29:76:ff:11 [ether] on ens33
? (192.168.10.50) at 00:00:00:01:02:03 [ether] PERM on ens33
ergs@ub-srv:~$ _

```

Al hacer ping la máquina, ya no nos sale el mensaje que nos informa que la red es inaccesible, sino que ahora intenta transmitir las tramas ethernet a esta MAC ficticia que le hemos asignado. Sin embargo, a pesar de que en la caché de ARP tiene una entrada de destino, el ping sigue sin funcionar por que se va a quedar esperando sin respuesta (Timeout) los paquetes de una dirección que es imposible que responda.

Poisoning de ARP

Borramos la caché de ambas máquinas con “arp -d”

ergs1

Para ergs1, borramos la entrada de la ip ficticia colocando la dirección después del “-d”

```

ergs@ub-srv:~$ sudo arp -d 192.168.10.50
[sudo] password for ergs:
ergs@ub-srv:~$ arp -a
? (192.168.10.2) at 00:0c:29:76:ff:11 [ether] on ens33
ergs@ub-srv:~$

```

Verificamos que con “arp -a” ya no aparece.

ergs2

Hacemos lo mismo, pero con la dirección de ergs1

```

ergs@UBD64-VMware-Virtual-Platform:~$ arp -a
? (192.168.10.1) en 00:0c:29:8b:ab:59 [ether] en ens33
ergs@UBD64-VMware-Virtual-Platform:~$ sudo arp -d 192.168.10.1
ergs@UBD64-VMware-Virtual-Platform:~$ arp -a
ergs@UBD64-VMware-Virtual-Platform:~$

```

Después de haber echo esto, asociaremos la dirección ip falsa (192.168.10.50) a la dirección MAC de la ergs2 (00:0c:29:76:ff:11) con el comando que ya habíamos utilizado antes y lo comprobamos:

```

ergs@ub-srv:~$ sudo arp -s 192.168.10.50 00:0c:29:76:ff:11
ergs@ub-srv:~$ arp -a
? (192.168.10.2) at 00:0c:29:76:ff:11 [ether] on ens33
? (192.168.10.50) at 00:0c:29:76:ff:11 [ether] PERM on ens33
ergs@ub-srv:~$ ip a

```

(Notamos que en ambas entradas la dirección MAC no cambia porque sigue siendo la misma que la de ergs2)

Ahora hacemos ping a esa IP:


```

ergs@ub-srv:~$ ping -c 4 192.168.10.50
PING 192.168.10.50 (192.168.10.50) 56(84) bytes of data.

--- 192.168.10.50 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3084ms

ergs@ub-srv:~$

```

A realizar el ping vemos que nuevamente ha fallado, pero si nos fijamos en el sniffer que tenemos en ergs2, los paquetes del ping sí que habrán llegado a la tarjeta de red.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=1/256, ttl=64 (no response found!)
2	0.066708059	0.0.0.0	255.255.255.255	DHCP	341	DHCP Discover - Transaction ID 0x879a432a
3	1.012523786	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=2/512, ttl=64 (no response found!)
4	2.036524895	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=3/768, ttl=64 (no response found!)
5	3.066197649	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=4/1024, ttl=64 (no response found!)
6	4.084416504	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=5/1280, ttl=64 (no response found!)
7	5.108213997	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=6/1536, ttl=64 (no response found!)
8	6.132324001	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=7/1792, ttl=64 (no response found!)
9	7.156181436	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=8/2048, ttl=64 (no response found!)
10	8.180207024	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=9/2304, ttl=64 (no response found!)
11	9.204109135	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=10/2560, ttl=64 (no response found!)
12	10.228199495	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=11/2816, ttl=64 (no response found!)
13	11.252184813	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=12/3072, ttl=64 (no response found!)
14	12.276581956	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=13/3328, ttl=64 (no response found!)
15	13.300191113	192.168.10.1	192.168.10.50	ICMP	98	Echo (ping) request id=0x1404, seq=14/3584, ttl=64 (no response found!)

Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface ens33
 Ethernet II, Src: VMware_8b:ab:59 (00:0c:29:8b:ab:59), Dst: VMware_76:ff:11 (00:0c:29:76:ff:11)
 Internet Protocol Version 4, Src: 192.168.10.1, Dst: 192.168.10.50
 Internet Control Message Protocol

Esto sucede porque a realizar el ping a la IP 192.168.10.50, ergs1 consulta su tabla ARP y no encuentra una entrada válida vinculada a la MAC de nuestra máquina ergs2. Entonces, ergs1 encapsula el paquete ICMP y lo envía físicamente hacia esa dirección que pertenece a ergs2. Si en nuestra máquina de Ubuntu Server no funciona el ping es porque físicamente la trama es correcta, pero ergs2 lo descarta ya que al procesar el paquete en la capa de red observa que la IP de destino es la .50, y como esa IP no está configurada en sus interfaces, el sistema operativo descarta el paquete por no ser el destinatario legítimo.

Consideraciones finales

La práctica ha resultado fundamental para entender que la comunicación depende tanto del uso y configuración de las direcciones IP, sino también de la resolución física (MAC). He aprendido cómo el protocolo ARP gestiona estas tablas de forma invisible y lo vulnerable que es ante técnicas de envenenamiento o *poisoning*, donde se puede desviar tráfico engañando a la caché usando conocimientos básicos. La parte más interesante de trabajar ha sido ver en Wireshark como los paquetes viajaban a un destino equivocado tras manipular la tabla. Lo más difícil fue la configuración inicial de red en Linux, mediante comandos de diagnóstico de rutas e interfaces.